Btrfs - copy-on-write B-tree filesystem

HISTORY / ORIGINAL DESIGN / DISK FORMAT

A Linux-native, checksummed, copy-on-write filesystem with subvolumes, snapshots and integrated multi-device logical-to-physical mapping.

This is a technical reference to the upstream filesystem's historical design, not an installer manual. The format's on-media semantics are distinct from the independent InfiltratorOS module, its APIs and any Linux kernel adapter.

CONCEPTUAL MEDIA MAP

CONTENTS

1. Historical chronology
2. Original on-disk architecture
3. Allocation and directories
4. Integrity and recovery
5. Design rationale and limitations
6. InfiltratorOS integration boundary

Btrfs - copy-on-write B-tree filesystem

HISTORICAL CHRONOLOGY

Oracle Linux developer Chris Mason began Btrfs development in 2007 as an advanced Linux filesystem with copy-on-write, checksums, snapshots and integrated volume management. Oracle’s project history records his June 2007 public announcement, and the implementation reached Linux mainline in the 2.6.29 cycle in 2009. The design addressed growing server storage complexity and features that simple journalling inode filesystems did not provide.

Btrfs evolved through widespread Linux contributions, new profiles, device replacement, send/receive and scrub facilities. Its implementation and recommended deployment profiles have changed considerably over time. Historical development claims should be separated from the state of a particular released kernel and userspace toolset.

Btrfs - copy-on-write B-tree filesystem

ORIGINAL ON-DISK ARCHITECTURE

The Btrfs primary superblock is at physical offset 64 KiB, with additional mirrors at specified offsets when the device is large enough. The superblock includes generation and pointers into a forest of B-trees. Important trees record filesystem roots, inodes and directory entries, extents, checksums, devices and chunk mappings. The format is transactional copy-on-write, so new tree versions are written rather than editing live ancestors in place.

Logical addresses are mapped through chunk structures to one or more physical devices and stripes. File extents and metadata blocks are not necessarily adjacent, and physical device addressing is not identical to the filesystem logical address space. Checksums are stored separately from the corresponding data in checksum trees. A parser must validate block headers, expected owner IDs and generation numbers.

Btrfs - copy-on-write B-tree filesystem

ALLOCATION AND DIRECTORIES

Space is managed through extent allocation and back-references rather than a FAT. Copy-on-write and reflinks permit multiple logical file references to share existing data extents. Subvolumes have separate filesystem roots and can be snapshotted; snapshots hold references to retained trees instead of copying an entire device. A changed shared extent may require allocating new data, metadata updates and later reference-count cleanup.

Inode items, directory items and directory index items belong to keyed trees. Lookup and enumeration must respect tree order and explicit keys. Compression is applied at extent boundaries and demands length and algorithm checks. Storage profiles such as single, DUP, RAID1 and RAID10 change placement and redundancy rules; they must not be inferred from the number of block devices alone.

Btrfs - copy-on-write B-tree filesystem

INTEGRITY AND RECOVERY

Btrfs checksums file data and metadata where applicable. Tree transactions commit through updated roots and superblock generation, so an interrupted operation may leave earlier committed roots valid. Scrub detects checksum mismatches and can repair damaged blocks if redundancy supplies another valid copy. Checksums without redundant storage detect corruption but cannot necessarily correct it.

Recovery requires checking generation, tree ownership, chunk mapping and device UUID identity. It is unsafe to rewrite trees from one superblock generation while using allocation state from another. RAID-related recovery and reshape operations involve complex profile-specific rules. Metadata consistency, data checksum errors and application durability are distinct questions.

Btrfs - copy-on-write B-tree filesystem

DESIGN RATIONALE AND LIMITATIONS

Btrfs combines filesystem and volume-management concerns to support flexible multi-device storage and inexpensive snapshots. This adds tree and reference bookkeeping, potential write amplification under random updates, and fragmentation of copy-on-write workloads. Full devices leave less working space for metadata updates and relocations.

Feature flags, compression algorithms, free-space-tree revisions and supported redundancy modes depend on the kernel and toolset. An implementation that reads basic trees does not automatically support scrub, rebuild, RAID maintenance or crash-recoverable writable mounts. Defragmentation may break extent sharing and increase storage use.

Btrfs - copy-on-write B-tree filesystem

INFILTRATOROS INTEGRATION BOUNDARY

BTRFS.so is a consumer module, not an upstream Linux Btrfs kernel replacement. Its advertised features should be restricted to verified parsing, analysis, writing and recovery operations. The user should be able to distinguish data checksum validation from metadata traversal and from device-level redundancy.

The authoritative Btrfs on-disk specification controls superblock copies, tree block headers, key semantics, chunk mapping and generation checks. Publishing a PDF or validating an ELF module does not establish multi-device write qualification.

Btrfs - copy-on-write B-tree filesystem

SOURCES AND PRIMARY REFERENCES

The documents below are the historical and engineering reference points used for this guide. Where an on-media field or feature flag is ambiguous, consult the original format documentation and its version-specific implementation notes before interpreting or changing media.

1. Btrfs official on-disk format reference

https://btrfs.readthedocs.io/en/latest/dev/On-disk-format.html

2. Btrfs historical contributions and merge chronology

https://btrfs.readthedocs.io/en/latest/Contributors.html

3. Oracle open-source Btrfs project history

https://opensource.oracle.com/ossh.html

The InfiltratorOS shared-object module, optional kernel adapter, native builder and release-specific tests are a separate implementation. A successful document build or package installation does not qualify physical writes, recovery, kernel mounting or boot.

Btrfs / engineering detail

DISK READ PATH / ENGINEERING NOTES

1. Start with the primary superblock at 64 KiB and validate its magic, checksum, generation and root pointers. Optional mirrors may exist at later device offsets.

2. Use system chunk mappings to bootstrap logical-to-physical addressing. A logical Btrfs address is not necessarily the same as a byte offset on one disk.

3. Read the root tree and chunk tree before interpreting individual filesystem trees; object IDs, item types and offsets form ordered B-tree keys.

4. Follow the extent tree to understand allocated extents and references. Several files, snapshots or subvolumes can legitimately refer to shared data.

5. Resolve a pathname through the appropriate subvolume tree, inode items, directory items and file extent items rather than a single universal inode table.

6. Validate checksums and copy-on-write generations for tree blocks. A valid-looking block from an old generation may not belong to the chosen root.

7. Interpret data and metadata block-group profiles separately, including single, duplicated, mirrored and parity-based storage policies.

8. Distinguish compressed extents, holes, inline data and regular extents; the logical file size is not necessarily the stored byte count.

Btrfs / engineering detail

VALIDATION, RECOVERY AND COMPATIBILITY

1. Choose a consistent root generation before walking trees. Mixing different committed generations can create a contradictory namespace and free-space view.

2. Validate chunk-to-device mappings and device identities; the correct logical extent may have several physical copies or stripes.

3. Check node key ordering, child generations, backreferences and extent reference counts before treating an extent as free.

4. An ordinary Btrfs data checksum protects a recorded extent, not necessarily all possible metadata damage or uncommitted writes.

5. Do not assume snapshots duplicate data blocks. Copy-on-write and backreferences allow shared extents until a modification requires divergence.

6. Treat a missing device, a failed mirror and a parity profile as different recovery conditions. Readability of one tree does not prove redundancy is healthy.

7. Unknown incompatibility flags and newer tree features must prevent unsafe writes from older implementations.

8. Separate a readonly rescue or scrub operation from metadata repair; each has different effects on the surviving evidence.